

# Training Budget and Time Parameterisation Decide the Ranking of Propagation Operators on LEO Inter-Satellite-Link Graphs

Phuc Hao Do<sup>a,\*</sup>

<sup>a</sup>*CAIRA, Danang Architecture University, Da Nang, Viet Nam*

---

## Abstract

Learned propagation on the inter-satellite-link graph of a low-Earth-orbit constellation is increasingly proposed for routing, state estimation and resource management, and several recent works argue that operators derived from continuous-time quantum walks reach farther than diffusion and should therefore help most on large graphs. We test that argument under controlled conditions. Three propagation operators that share one Laplacian eigenbasis and one network, a local GCN step, the heat kernel  $e^{-t\mathbf{L}}$  and the quantum-walk probability kernel  $|e^{-it\mathbf{L}}|^2$ , are compared on Walker-delta shells of 264 to 4400 satellites, on two node-field tasks, over ten seeds, and across arms that vary the parameterisation and initialisation of the learned propagation time  $t$ , the network capacity and the training budget, for 1920 plus 360 training runs on one machine. Three findings follow. First, with a training budget held fixed across scales, no operator improves on a constant predictor by more than 2.8% on any shell of 1584 satellites or more, in any arm: the comparison there measures a budget floor, not an operator. Second, on the one scale where learning occurs under that budget, the ranking of the two spectral operators changes with how  $t$  is parameterised and initialised, and traces to a seed-level bimodality in which a run either leaves the constant solution or does not. Third, with a five-fold budget every shell leaves the floor, and wherever the comparison is decidable the heat kernel matches or beats the walk kernel, with seed variance several times smaller; the walk kernel wins robustly in none of 6 task-scale cells. Long training at a fixed learning rate also collapses 11.1% of runs back to the constant solution. A preliminary version of this study reported a walk-kernel advantage at 1584 satellites from 3 seeds; that result is not reproduced, and its released artifact used a different configuration from its text. All code, results and the declared protocol are released.

*Keywords:* LEO constellations, inter-satellite links, graph neural networks, spectral propagation, quantum walks, evaluation methodology, reproducibility

---

## 1. Introduction

The inter-satellite-link (ISL) graph of a low-Earth-orbit (LEO) mega-constellation is large, nearly regular, and changes with time as satellites move and links across the seam and the polar regions come and go [1, 2]. Learned models that propagate information over this graph have

---

\*Corresponding author.

Email address: [haodp@dau.edu.vn](mailto:haodp@dau.edu.vn) (Phuc Hao Do)

been proposed for routing [3, 4], connectivity and multi-connectivity decisions [5], and related network functions. Most of them are graph neural networks, and a graph neural network is, at each layer, a propagation operator  $\Phi(\mathbf{L})$  applied to node features followed by a shared nonlinearity [6, 7]. Which operator to use is therefore a design decision with consequences at constellation scale.

Three operators span the design space that this paper studies. The local step of a graph convolutional network [6], equation (1) below, moves information one hop per layer. The heat kernel  $e^{-t\mathbf{L}}$  moves it diffusively, over a range that grows like  $\sqrt{t}$  [8]. The continuous-time quantum walk [9] evolves  $e^{-it\mathbf{L}}$  unitarily and spreads ballistically, over a range that grows like  $t$ ; its transition-probability kernel  $|e^{-it\mathbf{L}}|^2$  is a real, row-stochastic matrix that can be dropped into the same layer in place of the heat kernel [10, 11]. Recent work has turned the ballistic property into an argument: because the unitary propagator damps no frequency and preserves feature norm, walk-based operators should resist over-smoothing and help most where geodesics are long [12, 13, 14]. LEO ISL graphs, whose diameter grows with the number of satellites, are a natural place to look for that benefit.

A preliminary version of this study, circulated for review in June 2026 and available in the companion repository, looked and reported finding it: on a 1584-satellite shell the walk kernel reached a mean absolute error of 0.148 against 0.168 for the heat kernel, from 3 seeds, and the trend from smaller shells was read as the ballistic mechanism emerging with scale. This paper asks whether that ranking is a property of the operators or of the evaluation, and answers by controlling the evaluation.

The reason to suspect the evaluation is that three quantities were left implicit in the preliminary comparison and in most comparisons of this kind. The first is the training budget: if the number of epochs is held fixed while the graph grows, a model may simply not have left the constant solution on the large graph, and a comparison between two operators that have both failed to learn measures noise. The second is the learned propagation time  $t$ : both spectral operators carry a per-layer  $t$  that is trained by gradient descent, and how it is parameterised and where it starts determines how far each operator can reach and whether it can reach at all. The third is the number of seeds, which decides whether a difference is distinguishable from run-to-run variation. We make all three explicit and vary them.

The contribution is a controlled comparison and what it shows.

- A protocol in which the three operators share one eigendecomposition, one network, one parameter count and one training loop, and in which the training budget, the  $t$  parameterisation and initialisation, the network capacity and the seed count are arms of the study rather than fixed choices. Every rule for reading the results, including the constant baseline, the floor below which a cell is not rankable, the seed-majority needed to call a winner, and the requirement that a ranking hold in every arm, was declared before the runs and is recorded in the repository history.
- Under a fixed budget of 200 epochs, no operator improves on the constant predictor by more than 2.8% on any shell of 1584 satellites or more, in any of eight configuration-by-arm combinations (Section 5.1). On the one shell where learning occurs, the heat-versus-walk ranking changes with the  $t$  arm, and the reason is visible at the seed level: a run either leaves the constant solution or stays on it (Section 5.2).
- With a budget of 1000 epochs every shell leaves the floor. Wherever the comparison is decidable under the declared rules, the heat kernel matches or beats the walk kernel, with

seed variance several times smaller, and the walk kernel wins in none of 6 task-scale cells (Section 5.3). Long training at a fixed learning rate collapses 11.1% of runs back to the constant solution, which the protocol counts rather than drops.

- The preliminary result is not reproduced under this protocol, and the released artifact of the preliminary version turns out to have used a network of 3301 parameters while its released code defaults to 612, with no script for the experiment in question (Section 7). We report this because it is the kind of gap the protocol is designed to close.

The rest of the paper is organised as follows. Section 2 places the operators and the LEO setting in the literature. Section 3 defines the graphs (Section 3.1), the three operators on one eigenbasis (Section 3.2) and the network and tasks (Section 3.3). Section 4 states the protocol and the reading rules (Section 4.2). Section 5 reports the three grids. Section 6 interprets them, Section 7 lists what limits them, and Section 8 concludes.

## 2. Related work

*Propagation operators and their reach.* Spectral graph convolutions began as polynomial filters of the Laplacian [15, 7]; the graph convolutional network of Kipf and Welling [6] keeps the first-order term and applies a normalised adjacency once per layer, so depth bounds reach. Stacking layers drives features toward a constant, the over-smoothing effect identified by Li et al. [16], shown by Oono and Suzuki [17] to be exponential in depth, and surveyed by Rusch et al. [18]; remedies include decoupling propagation from transformation as in SGC [19] and personalised-PageRank propagation [20], and residual and identity terms as in GCNII [21]. Alon and Yahav [22] describe the complementary bottleneck by which information from exponentially many distant nodes is squashed into fixed-size vectors, which Topping et al. [23] relate to negative curvature of the graph. Continuous-time views replace the discrete step with a differential equation on the graph: GRAND [8] uses diffusion, whose solution is the heat kernel; PDE-GCN [24] and graph-coupled oscillator networks [25] use wave-type dynamics, whose reach grows linearly in time rather than as a square root. The distinction between diffusive and ballistic spreading that motivates the walk kernel in this paper is the same distinction, and the wave-type models are its classical form.

*Quantum walks as graph operators.* The continuous-time quantum walk of Farhi and Gutmann [9] evolves an amplitude vector by  $e^{-it\mathbf{L}}$  and spreads ballistically; Kempe [26] and Mülken and Blumen [27] review the walk and its transport properties on complex networks, and Childs [28] shows it is computationally universal. Dernbach et al. [10] and Verdon et al. [11] were among the first to place walk-derived operators inside trainable graph models. GraphQWalk [14] builds unsupervised, position-independent structural embeddings from walk transition probabilities. CTQWformer [13] learns the walk Hamiltonian and feeds final-time probabilities into a graph transformer. CTQW-GNN [12] uses the unitary propagator itself in feature space and proves that, because its eigenvalues lie on the unit circle, no frequency is damped and the Dirichlet energy of the features cannot decay exponentially with depth. That result concerns the unitary operator acting on features. The operator studied here is the transition-probability kernel  $|e^{-it\mathbf{L}}|^2$ , a real nonnegative matrix that shares the ballistic profile of the walk but not the norm-preserving property; the distinction matters for interpreting the results (Section 6). Quantum walks on communication networks have also been used directly, for instance the Hadamard-walk model for network identification of Liang et al. [29].

Table 1: The four shells. All use +Grid links, a  $70^\circ$  polar cutoff and a seam cut. The two largest are scale-ups of the Starlink shell-1 geometry and are not deployed constellations.

| satellites | planes | per plane | incl. ( $^\circ$ ) | alt. (km) | note                                  |
|------------|--------|-----------|--------------------|-----------|---------------------------------------|
| 264        | 24     | 11        | 53                 | 550       | main scale of the preliminary version |
| 1584       | 72     | 22        | 53                 | 550       | Starlink shell-1 geometry [2]         |
| 3168       | 72     | 44        | 53                 | 550       | scale-up, synthetic                   |
| 4400       | 100    | 44        | 53                 | 550       | scale-up, synthetic                   |

*Graph learning on LEO constellations.* Handley [30] first argued that inter-satellite routing in a mega-constellation is a latency question, Bhattacharjee and Singla [1] fix the +Grid inter-satellite topology that most later work assumes, Kassing et al. [31] provide the Hypatia simulator for such networks, and del Portillo et al. [2] compare the announced constellations, including the Starlink shell whose geometry the 1584-satellite graph here reproduces. Learned models on this graph include the link-identified routing architecture of Zhang et al. [3], the duality-guided graph learning for joint connectivity and routing of Gu et al. [4], and machine-learning-driven cellular-satellite multi-connectivity [5]. None of these compares propagation operators under a common parameterisation of reach, which is the gap this study addresses. The tasks used here are deliberately decoupled from any particular routing or traffic-engineering system, so that the propagation question is isolated from questions of load.

*Evaluation pitfalls.* Shchur et al. [32] showed that the ranking of graph neural network architectures on standard benchmarks changes with the data split and the seed, Henderson et al. [33] made the corresponding point for deep reinforcement learning, where few seeds and unreported tuning produce rankings that do not replicate, Bouthillier et al. [34] quantify how much benchmark variance comes from sources other than the method, and Pineau et al. [35] report on the reproducibility programme that turned such findings into checklists. The present study applies the same discipline to a comparison of propagation operators, and adds two quantities specific to spectral operators, the training budget relative to graph size and the parameterisation of the learned propagation time.

### 3. Setting

#### 3.1. Constellation graphs

Each graph is a snapshot of a Walker-delta shell. Satellite positions follow circular-orbit propagation in an Earth-centred inertial frame; inter-satellite links follow the +Grid pattern [1], two intra-plane and two inter-plane links per satellite, with inter-plane links suppressed above  $70^\circ$  latitude and across the counter-rotating seam. The graph is restricted to its largest connected component. Four shells are used (Table 1). The 264-satellite shell is the main scale of the preliminary version; the 1584-satellite shell reproduces the geometry of the first Starlink shell [2], 72 planes of 22 satellites at  $53^\circ$  and 550 km; the 3168- and 4400-satellite shells are scale-ups of that geometry, 72 planes of 44 and 100 planes of 44, and are synthetic in the sense that no operator flies them. They exist to make the diameter grow.

Each training or evaluation instance draws a snapshot time uniformly over the orbital period and a destination satellite uniformly over the component. Instance construction is seeded, so every instance is reproducible from its index.

### 3.2. Three operators on one eigenbasis

Let  $\mathbf{L} = \mathbf{D} - \mathbf{A}$  be the combinatorial Laplacian of the snapshot, with eigendecomposition  $\mathbf{L} = \mathbf{U} \text{diag}(\lambda) \mathbf{U}^\top$  computed once per instance and shared by all operators. The three operators are

$$\Phi_{\text{gcn}} = \tilde{\mathbf{D}}^{-1/2}(\mathbf{A} + \mathbf{I})\tilde{\mathbf{D}}^{-1/2}, \quad (1)$$

$$\Phi_{\text{heat}}(t) = \mathbf{U} \text{diag}(e^{-t\lambda}) \mathbf{U}^\top = e^{-t\mathbf{L}}, \quad (2)$$

$$\Phi_{\text{walk}}(t) = \left| \mathbf{U} \text{diag}(e^{-it\lambda}) \mathbf{U}^\top \right|^2 = |e^{-it\mathbf{L}}|^2, \quad (3)$$

where the square in (3) is taken entrywise on the modulus, so that  $[\Phi_{\text{walk}}]_{ij}$  is the probability that a continuous-time quantum walk started at  $j$  is found at  $i$  after time  $t$ . Both (2) and (3) are real, nonnegative and row-stochastic; they differ in the profile of the row as a function of graph distance, monotone for the heat kernel and oscillatory for the walk kernel, and in how that profile's width grows with  $t$ .

### 3.3. Network, parameterisation of $t$ , and tasks

The network has  $K$  layers, each computing  $\mathbf{H}^{(\ell+1)} = \sigma(\Phi^{(\ell)} \mathbf{H}^{(\ell)} \mathbf{W}^{(\ell)})$  with a ReLU nonlinearity and a linear read-out to one scalar per node. For the two spectral operators, each layer carries its own scalar propagation time  $t_\ell$ , trained jointly with the weights. The preliminary version parameterised it as  $t_\ell = \text{relu}(\theta_\ell)$  with  $\theta_\ell$  initialised at 0.5. Section 4 explains why this is an arm of the present study and adds  $t_\ell = \text{softplus}(\theta_\ell)$ , which keeps  $t_\ell > 0$  with a live gradient, initialised so that the two parameterisations start at the same value. The GCN operator has no  $t$ .

The input to every node is two features, an indicator of the destination and the normalised degree. Two node-field targets are used. With  $d$  the destination,  $h(i, d)$  the geodesic hop count and  $\tau(i, d)$  the shortest-path propagation delay computed on the inter-satellite link lengths, the targets are

$$y_i^{\text{hop}} = \frac{h(i, d)}{\max_j h(j, d)}, \quad y_i^{\text{delay}} = \frac{\tau(i, d)}{\max_j \tau(j, d)}, \quad (4)$$

so that both lie in  $[0, 1]$  and the constant predictor has the same meaning on every shell. The hop field is the task of the preliminary version. Both fields in (4) require the model to move a single seeded signal across the graph, which is the propagation question in isolation; neither involves traffic or load. The metric is the mean absolute error (MAE) over nodes on held-out instances.

## 4. Protocol

### 4.1. Parameter matching and the arms

The three operators share depth, width, weight shapes and training loop, so a difference in error is attributable to the operator and not to capacity. Two capacities are used: width 16 with three layers, which is the default of the released code of the preliminary version (612 parameters including the  $t_\ell$ ), and width 32 with four layers, which the text of the preliminary version describes and its released results were produced with (3301 parameters). Adam [36] at learning rate 0.01, eight training and six evaluation instances per seed, and full-batch updates are used throughout.

Four quantities are arms rather than fixed choices.

- **Parameterisation of  $t$ .** A pilot run under the released parameterisation showed the learned  $t_\ell$  going negative in the second and third layers, where  $\text{relu}(t_\ell) = 0$  turns the propagator into the identity and stops its gradient, so that a model reported as three or four layers of spectral propagation was propagating in one. The softplus parameterisation removes this; the relu parameterisation is kept as a control so that the preliminary configuration is one of the arms.
- **Initialisation of  $t$ .**  $t_0 \in \{0.5, 2\}$ , since the same pilot showed the learned reach, and with it the operator ranking, depending on where  $t$  starts.
- **Capacity.** The two configurations above.
- **Budget.** 200 epochs, the preliminary value, held fixed across scales; and 1000 epochs for the second grid.

Grid 1 runs both capacities, all four  $t$  arms, four shells, two tasks, three operators and ten seeds at 200 epochs (1920 runs). Grid 2 runs the width-32 configuration at 1000 epochs on the three smaller shells with the two  $t_0 = 0.5$  arms (360 runs). All runs were executed on one machine, one GPU per arm, with the eigendecomposition of each instance cached and shared so that every arm sees bit-identical eigenpairs.

#### 4.2. Reading rules, declared before the runs

Five rules govern how a cell, one (task, shell, arm) combination, is read. They were fixed before the first grid was launched, and two of them were tightened after the first runs exposed a gap; both changes are dated in the repository and stated here.

1. **Constant baseline.** Every cell reports the MAE of the predictor that outputs the mean of the target. Whether an operator beats it is a result, never a filter.
2. **Pathology.** A run is excluded only for a training pathology: a non-finite loss, or a loss that rises by more than 1% between successive thirds of training. Excluded runs are counted next to every number they affect. A run whose loss is flat from the start because it sits on the constant solution is a *plateau*, a legitimate outcome, and is kept. The first draft of this rule required a 10% loss drop and would have discarded plateaus as failures; the draft was corrected when the first grid produced them.
3. **Floor.** Let  $\bar{e}_{\text{op}}$  be the mean MAE of an operator over its valid seeds in a cell and  $\bar{e}_0$  that of the constant predictor. The cell's gain is

$$g = \frac{\bar{e}_0 - \min_{\text{op}} \bar{e}_{\text{op}}}{\bar{e}_0}, \quad (5)$$

and the cell is *rankable* only if  $g \geq 0.05$ . Below that, differences between operators are within the band in which no operator has learned, and the cell is reported as a floor. This rule was added after the first completed cells, all on the 4400-satellite shell, showed the three operators' means within 0.002 of one another and of the constant baseline while the seed-majority rule below still declared a winner.

4. **Winner.** Within a rankable cell, with  $S$  the set of seeds valid for both spectral operators and  $e_{\text{op},s}$  the MAE of seed  $s$ , the heat kernel wins if

$$\bar{e}_{\text{heat}} < \bar{e}_{\text{walk}} \quad \text{and} \quad |\{s \in S : e_{\text{heat},s} < e_{\text{walk},s}\}| \geq 7, \quad (6)$$

and symmetrically for the walk kernel; otherwise the cell is a tie. Fewer than seven valid seed pairs,  $|S| < 7$ , leaves the cell undecided.

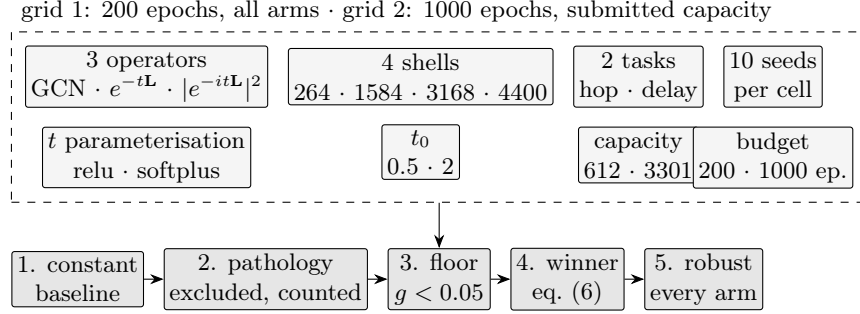


Figure 1: The study design. Every cell of the grid, one (task, shell, arm), runs the three operators on the same instances and seeds; the five reading rules are applied in order by one script, and a ranking is reported only if it survives all five.

Table 2: Headline numbers and where each is derived. Every value is generated from the result files by the same script that produces the figures.

| finding                                                                   | value                   | derived in     |
|---------------------------------------------------------------------------|-------------------------|----------------|
| runs, fixed 200-epoch budget (grid 1)                                     | 1920                    | Sec. 5.1       |
| largest gain over constant on any shell $\geq 1584$ , any arm (grid 1)    | 2.9%                    | Fig. 2, Tab. 3 |
| rankable cells at $\geq 1584$ satellites, hop field (grid 1)              | 0 of 24                 | Tab. 3         |
| heat-vs-walk verdicts at 264 satellites across the four $t$ arms (grid 1) | tie / heat / tie / heat | Fig. 3         |
| runs, 1000-epoch budget (grid 2)                                          | 360                     | Sec. 5.3       |
| task-scale cells where heat wins in every arm (grid 2)                    | 3 of 6                  | Tab. 4         |
| task-scale cells where walk wins in every arm (grid 2)                    | 0 of 6                  | Tab. 4         |
| runs collapsing to the constant solution under long training (grid 2)     | 40 (11.1%)              | Sec. 5.4       |
| preliminary 1584-satellite result, walk vs heat, 3 seeds                  | 0.148 vs 0.168          | Sec. 5.3       |

5. **Robustness.** A ranking at a (task, shell) is claimed only if it is the same in every  $t$  arm run at that budget.

The rules are implemented once, in the analysis script that produces every table and figure, and the same script prints the pathology and plateau counts beside every mean. Figure 1 summarises the grids and the reading rules, and Table 2 collects the headline numbers with the section that derives each.

## 5. Results

### 5.1. A fixed budget produces a floor at scale

Figure 2 and Table 3 show, for each capacity and each  $t$  arm, the gain (5) of the best operator over the constant predictor as the shell grows, at the preliminary budget of 200 epochs. On the

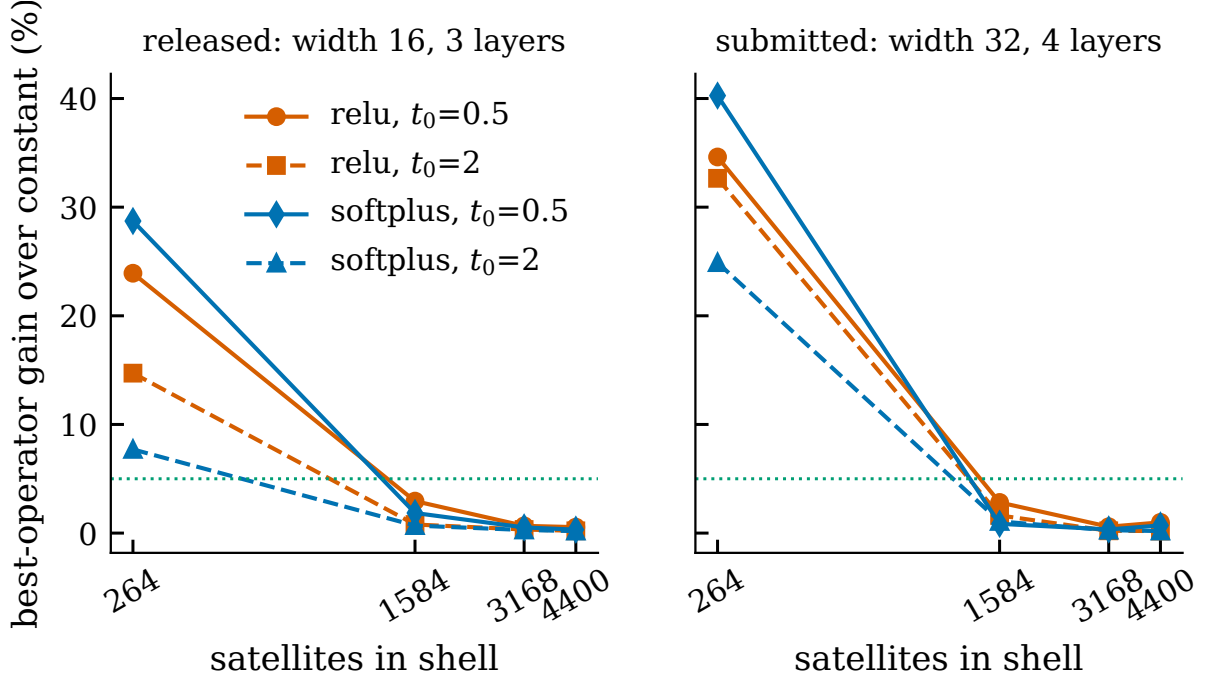


Figure 2: Improvement of the best operator over the constant predictor on the hop field at 200 epochs, as a function of shell size, for the released and the submitted capacity and the four  $t$  arms. The dotted line is the 5% floor of rule 3. Above 264 satellites every combination lies below it.

Table 3: Improvement (%) of the best operator over the constant predictor, hop field, 200 epochs, ten seeds per cell. A dagger marks cells below the 5% floor, where the operator ranking is not read.

| config    | $t$ arm             | 264  | 1584             | 3168             | 4400             |
|-----------|---------------------|------|------------------|------------------|------------------|
| released  | relu, $t_0=0.5$     | 23.9 | 2.9 <sup>†</sup> | 0.7 <sup>†</sup> | 0.5 <sup>†</sup> |
| released  | relu, $t_0=2$       | 14.7 | 0.8 <sup>†</sup> | 0.3 <sup>†</sup> | 0.2 <sup>†</sup> |
| released  | softplus, $t_0=0.5$ | 28.7 | 1.8 <sup>†</sup> | 0.5 <sup>†</sup> | 0.3 <sup>†</sup> |
| released  | softplus, $t_0=2$   | 7.7  | 0.7 <sup>†</sup> | 0.3 <sup>†</sup> | 0.2 <sup>†</sup> |
| submitted | relu, $t_0=0.5$     | 34.6 | 2.8 <sup>†</sup> | 0.6 <sup>†</sup> | 1.0 <sup>†</sup> |
| submitted | relu, $t_0=2$       | 32.7 | 1.6 <sup>†</sup> | 0.2 <sup>†</sup> | 0.2 <sup>†</sup> |
| submitted | softplus, $t_0=0.5$ | 40.3 | 0.9 <sup>†</sup> | 0.3 <sup>†</sup> | 0.7 <sup>†</sup> |
| submitted | softplus, $t_0=2$   | 24.9 | 1.1 <sup>†</sup> | 0.3 <sup>†</sup> | 0.2 <sup>†</sup> |

264-satellite shell the improvement is between 7.7% and 28.7% at the released capacity and between 24.9% and 40.3% at the submitted capacity. On every larger shell it never exceeds 2.8%, in any of the eight capacity-by-arm combinations, on either task: of the 24 hop-field cells at 1584 satellites and above, 0 are rankable. The models have not left the constant solution. Of 1920 runs, 5 were excluded as pathologies and 1352 on the large shells were plateaus in the sense of rule 2.

Doubling the width and adding a layer does not change this: the submitted capacity is as far below the floor at 1584 satellites as the released one. Nor does the  $t$  arm. The floor is therefore not a capacity effect and not a parameterisation effect; Section 5.3 shows that it is a budget effect.

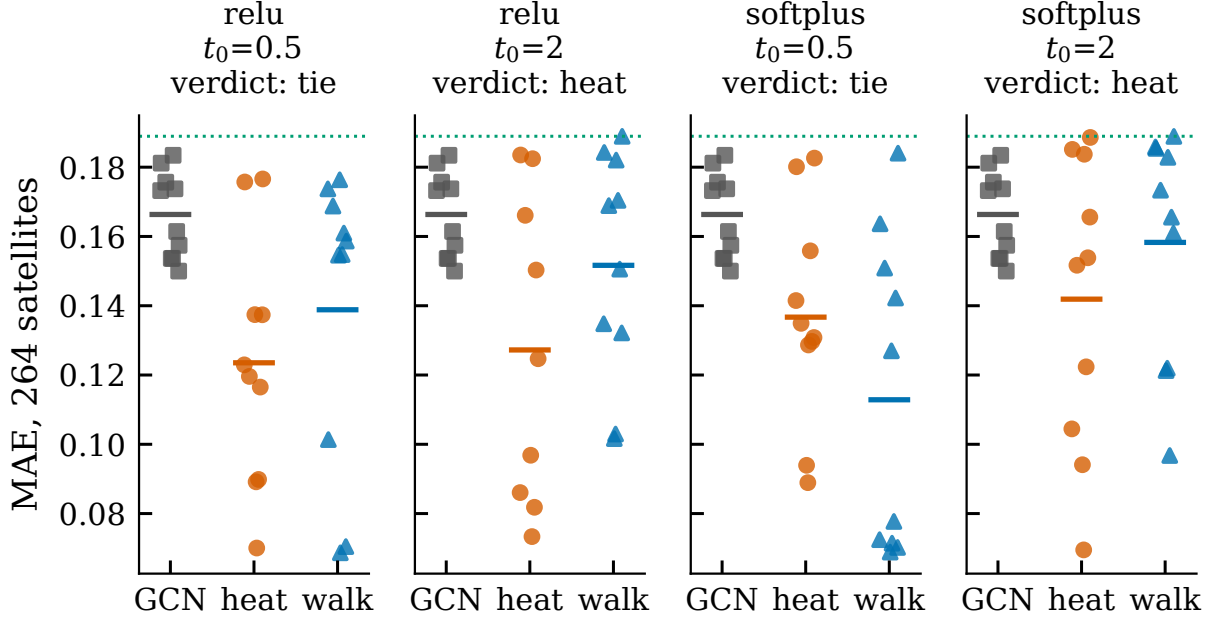


Figure 3: Per-seed MAE on the hop field, 264 satellites, submitted capacity, 200 epochs, for the four  $t$  arms. Bars are means over valid seeds; the dotted line is the constant predictor. The two spectral operators are bimodal across seeds: a run either leaves the constant solution or stays near it.

### 5.2. The propagation time is a hidden variable

On the 264-satellite shell, where learning does occur at 200 epochs, the ranking between the two spectral operators is not stable. Figure 3 plots every seed for the submitted capacity under the four  $t$  arms. The declared verdicts read, in arm order, tie / heat / tie / heat: the same operators on the same instances rank differently depending only on how  $t$  is parameterised and where it starts, so no ranking is claimed at this scale (rule 5).

The figure also shows why. For both spectral operators the ten seeds do not scatter around a mean; they split. Some runs reach errors well below the constant predictor and the rest stay near it. A ranking between two operators at this budget is largely a count of how many seeds escaped, and that count moves with the  $t$  arm because  $t$  decides how far a layer reaches at initialisation and how quickly gradient descent can move it. The left panel of Figure 4 shows the learned  $t_\ell$  per layer as a function of shell size for the softplus arm at this budget: the first layer learns a larger  $t$  than the later ones for both operators, and the values do not grow with the shell, which is consistent with the models on large shells not having moved from the constant solution within the budget. The right panel, discussed in Section 5.3, shows what happens when they do.

### 5.3. With enough budget, the heat kernel matches or beats the walk kernel

Grid 2 raises the budget to 1000 epochs at the submitted capacity on the three smaller shells. Every cell now leaves the floor: on the hop field the best operator improves on the constant predictor by 15% to 69% depending on shell and arm, and on the delay field by 20% to 36% (Table 4). The floor of Section 5.1 was a budget floor.

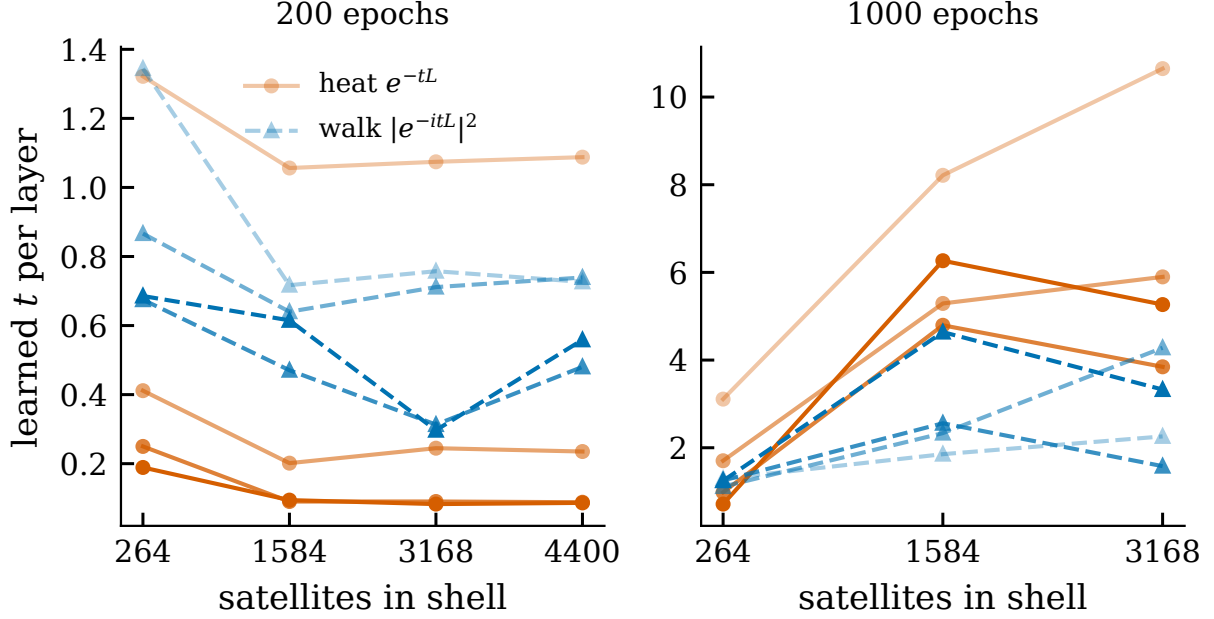


Figure 4: Learned propagation time per layer (lighter to darker: first to fourth layer) against shell size, hop field, submitted capacity, softplus arm with  $t_0 = 0.5$ , mean over valid seeds. Left: 200 epochs (grid 1). Right: 1000 epochs (grid 2). Note the different vertical scales.

Figure 5 and Table 4 give the comparison. Under rule 5, the heat kernel wins robustly at 264 satellites on both tasks and at 1584 satellites on the delay field: 3 of 6 task-scale cells. The remaining cells are ties or undecided because pathologies reduced the valid seed count below seven. The walk kernel wins robustly in 0 cells. Where the two are close in mean, they are not close in variance: on the hop field at 264 satellites in the softplus arm, the heat kernel reaches  $0.058 \pm 0.005$  against  $0.064 \pm 0.004$  for the walk kernel, with the constant predictor at 0.189; on the larger shells the walk kernel’s standard deviation across seeds is several times the heat kernel’s in most cells. The local GCN operator is the worst everywhere, as expected for a four-hop model on graphs whose eccentricity is tens of hops; on the hop field it clears the 5% floor only on the 264-satellite shell (gain 27.6%, against 4.5% at 1584 and 2.6% at 3168).

The right panel of Figure 4 shows what the budget buys. At 1000 epochs the heat kernel’s first-layer  $t$  grows with the shell, from 3.1 at 264 satellites to 8.2 at 1584 and 10.6 at 3168, while the walk kernel’s stays between 1.3 and 2.3. The diffusive operator reaches farther on larger graphs by learning to; the walk operator does not move its time parameter in the same way, and its larger seed variance is consistent with a loss that is not monotone in  $t$ .

The preliminary version’s finding, a walk-kernel error of 0.148 against 0.168 at 1584 satellites, is not reproduced. Under the same parameterisation and capacity, with ten seeds instead of 3, the two operators tie on the hop field at that scale and the heat kernel wins on the delay field.

#### 5.4. Long training collapses a tenth of the runs

Of the 360 runs of grid 2, 40 (11.1%) were excluded under rule 2. Inspection shows one pattern: the loss falls, then rises again by up to 36% and the run ends on the constant solution. It affects all

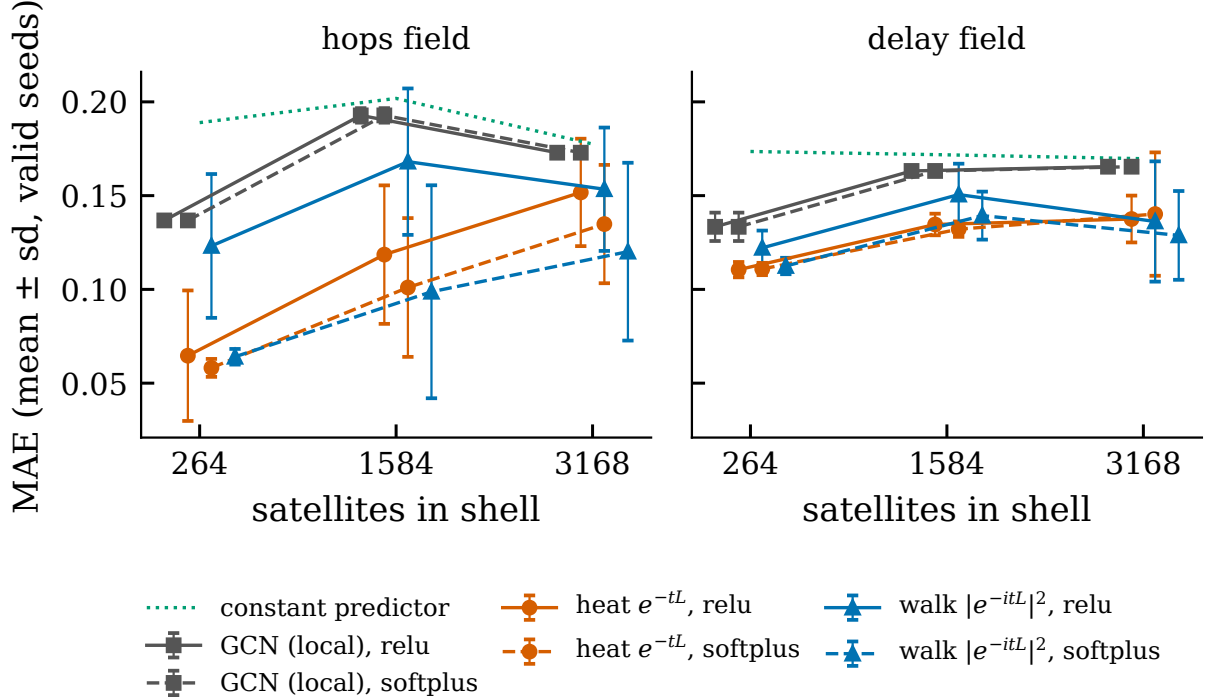


Figure 5: MAE against shell size at 1000 epochs, submitted capacity, both  $t$  arms, both tasks. Points are means over valid seeds and bars one standard deviation; the dotted line is the constant predictor. Points are offset horizontally within each shell for legibility.

three operators and both  $t$  arms, and it is the reason several cells in Table 4 fall below seven valid seed pairs. A fixed learning rate that is adequate for 200 epochs is not stable for 1000 on these graphs. We report the rate rather than tune it away, because a reader who runs a longer schedule to escape the floor of Section 5.1 should expect to meet it.

## 6. Discussion

*Why the probability kernel does not inherit the unitary argument.* The case for walk-based operators in CTQW-GNN [12] rests on the unitary propagator acting on feature vectors: it damps no eigenvector and preserves the Dirichlet energy of the features across layers. The kernel studied here, and in the preliminary version, is the transition-probability matrix (3). It is a real nonnegative row-stochastic matrix, exactly as the heat kernel is, and applying it to features damps and mixes them in the same way a diffusion step does. What it inherits from the walk is only the shape of its rows: an oscillatory profile whose width grows linearly with  $t$  rather than as a square root. Once  $t$  is a trained parameter, the heat kernel can reach as far by learning a larger  $t$ , which the right panel of Figure 4 shows it does, in the first layer and in proportion to the shell. The remaining difference is the profile, and an oscillatory profile makes the loss a non-monotone function of  $t$ , which is a plausible source of the larger seed variance the walk kernel shows in Table 4. A fair test of the unitary argument on LEO graphs would use the propagator in feature space, as CTQW-GNN does, and is a different study.

Table 4: Grid 2: MAE at 1000 epochs, submitted capacity, mean  $\pm$  standard deviation over valid seeds with the count in parentheses, the number of seeds on which each spectral operator is lower, and the verdict under rules 3 and 4. “n<7” marks a cell with too few valid seed pairs to decide.

| task                                               | shell | GCN   | heat $e^{-tL}$         | walk $ e^{-itL} ^2$    | seeds | heat:walk | verdict |
|----------------------------------------------------|-------|-------|------------------------|------------------------|-------|-----------|---------|
| <i>hops field, relu, <math>t_0=0.5</math></i>      |       |       |                        |                        |       |           |         |
|                                                    | 264   | 0.137 | $0.065 \pm 0.035$ (9)  | $0.123 \pm 0.038$ (10) |       | 8:1       | heat    |
|                                                    | 1584  | 0.193 | $0.119 \pm 0.037$ (6)  | $0.168 \pm 0.039$ (10) |       | 4:2       | n<7     |
|                                                    | 3168  | 0.173 | $0.152 \pm 0.029$ (9)  | $0.153 \pm 0.033$ (9)  |       | 5:3       | tie     |
| <i>hops field, softplus, <math>t_0=0.5</math></i>  |       |       |                        |                        |       |           |         |
|                                                    | 264   | 0.137 | $0.058 \pm 0.005$ (10) | $0.064 \pm 0.004$ (10) |       | 7:3       | heat    |
|                                                    | 1584  | 0.193 | $0.101 \pm 0.037$ (10) | $0.099 \pm 0.057$ (8)  |       | 2:6       | tie     |
|                                                    | 3168  | 0.173 | $0.135 \pm 0.032$ (7)  | $0.120 \pm 0.047$ (8)  |       | 2:3       | n<7     |
| <i>delay field, relu, <math>t_0=0.5</math></i>     |       |       |                        |                        |       |           |         |
|                                                    | 264   | 0.133 | $0.111 \pm 0.004$ (10) | $0.122 \pm 0.009$ (7)  |       | 7:0       | heat    |
|                                                    | 1584  | 0.163 | $0.135 \pm 0.006$ (10) | $0.151 \pm 0.016$ (10) |       | 8:2       | heat    |
|                                                    | 3168  | 0.165 | $0.138 \pm 0.012$ (10) | $0.136 \pm 0.032$ (10) |       | 6:4       | tie     |
| <i>delay field, softplus, <math>t_0=0.5</math></i> |       |       |                        |                        |       |           |         |
|                                                    | 264   | 0.133 | $0.111 \pm 0.003$ (9)  | $0.112 \pm 0.004$ (10) |       | 7:2       | heat    |
|                                                    | 1584  | 0.163 | $0.132 \pm 0.004$ (10) | $0.139 \pm 0.013$ (10) |       | 8:2       | heat    |
|                                                    | 3168  | 0.165 | $0.140 \pm 0.033$ (9)  | $0.129 \pm 0.024$ (7)  |       | 1:5       | n<7     |

*What the floor means for learned propagation on constellations.* A budget that suffices on a 264-satellite shell does not on a 1584-satellite one, and the failure is silent: the run reports a finite, decreasing loss and an error that looks like a number. Any comparison of learned propagation across constellation sizes must therefore carry the constant baseline beside every result and must let the budget grow with the graph, or it will rank noise. The same holds for comparisons across operator families in which one family carries a learned reach parameter and the other does not.

*What the seed-level bimodality means for reporting.* The split in Figure 3 means that a mean over three seeds at this budget is the mean of a coin flip. The preliminary version’s 3-seed result at 1584 satellites sits within the range that the bimodality produces, and its direction is the one that Section 5.3 does not confirm. Ten seeds and a seed-majority rule are the minimum that made the present cells decidable, and even then the pathology rate of Section 5.4 left several undecided.

## 7. Threats to validity

**The two largest shells are synthetic.** The 3168- and 4400-satellite graphs scale the Starlink shell-1 geometry and are not deployed constellations. They serve to grow the diameter; conclusions about them are conclusions about graph size, not about any operator’s network.

**The tasks are node fields, not network functions.** Hop and delay fields isolate propagation from load, which is what the comparison needs, at the cost of distance from any routing or traffic-engineering objective. A learned model on a real objective would sit on top of these operators, and the floor and variance effects would carry over; the ranking might not.

**One learning rate, one optimiser.** The collapse rate of Section 5.4 is specific to Adam at 0.01 for 1000 epochs. A schedule would likely reduce it and might change the undecided cells. The

protocol declares the rate rather than tunes it because tuning per operator would break parameter matching.

**The reading rules are choices.** The 5% floor and the seven-of-ten majority were fixed before the runs and would give different cell verdicts at other values. Both are reported with every table so a reader can apply their own.

**The preliminary artifact does not reproduce its own headline.** The released results of the preliminary version record 3301 parameters for the 1584-satellite experiment, which corresponds to the width-32, four-layer network its text describes, while the released code defaults to width 16 and three layers (612 parameters) and contains no script for that experiment. The preliminary version also ran without the seam cut that its text states. The present study fixes the seam cut on, runs both capacities, and releases the script for every number. We state this because the protocol of Section 4 exists to prevent exactly this class of gap, and it was found by applying the protocol to our own earlier work.

## 8. Conclusion

We asked whether the reported advantage of a quantum-walk probability kernel over the heat kernel on large LEO inter-satellite-link graphs is a property of the operator. Under a protocol that matches parameters, shares eigenbases, varies the parameterisation and initialisation of the learned propagation time, doubles the capacity, raises the budget and runs ten seeds with declared reading rules, it is not. At the preliminary budget the large shells are a floor on which no operator learns; at the one shell where learning occurs the ranking follows the  $t$  arm through a seed-level bimodality; and with a budget that clears the floor, the heat kernel matches or beats the walk kernel wherever the comparison can be decided, with smaller variance. The practical recommendations are short: carry the constant baseline, scale the budget with the graph, treat the propagation time as a declared arm, and count collapsed runs instead of dropping them.

### Data and code availability

All code, the cached graph instances’ construction, every result file of the 1920 plus 360 runs, the scripts that produce every figure, table and number in this paper, and the dated protocol declarations are available at <https://github.com/haodpsut/leo-propagation-reach>. The preliminary version and its released artifact are available at <https://github.com/haodpsut/qi-gnn-dynamic-wireless>.

### CRediT authorship contribution statement

**Phuc Hao Do:** Conceptualization, Methodology, Software, Investigation, Data curation, Writing, Visualization.

### Declaration of competing interest

The author declares no competing interests.

## Declaration of generative AI use

During the preparation of this work the author used a large language model (Claude, Anthropic) to assist with code, data analysis scripts and editing of the text. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

## References

- [1] D. Bhattacharjee, A. Singla, Network topology design at 27,000 km/hour, in: Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies, 2019, pp. 341–354. doi:10.1145/3359989.3365407.
- [2] I. del Portillo, B. G. Cameron, E. F. Crawley, A technical comparison of three low earth orbit satellite constellation systems to provide global broadband, *Acta Astronautica* 159 (2019) 123–135. doi:10.1016/j.actaastro.2019.03.040.
- [3] H. Zhang, Z. Wang, S. Zhang, Q. Meng, H. Luo, Link-identified routing architecture in space, *IEEE Transactions on Network Science and Engineering* 12 (1) (2025) 392–408. doi:10.1109/tnse.2024.3498042.
- [4] Z. Gu, J. Choi, T. Q. S. Quek, J. Park, Duality-guided graph learning for real-time joint connectivity and routing in leo mega-constellations, *arXiv preprint arXiv:2601.21921* (2026). arXiv:2601.21921.
- [5] P. Maurya, A. Ramírez-Arroyo, T. B. Sørensen, S. B. Damsgaard, Machine learning-driven cellular-satellite multi-connectivity for monitoring livestock transport in rural areas, *Computer Communications* 246 (2026) 108366. doi:10.1016/j.comcom.2025.108366.
- [6] T. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, *arXiv preprint arXiv:1609.02907* ArXiv (Cornell University) (2016). arXiv:1609.02907, doi:10.48550/arxiv.1609.02907.
- [7] D. I. Shuman, S. K. Narang, P. Frossard, A. Ortega, P. Vandergheynst, The emerging field of signal processing on graphs: Extending high-dimensional data analysis to networks and other irregular domains, *IEEE Signal Processing Magazine* 30 (3) (2013) 83–98. doi:10.1109/MSP.2012.2235192.
- [8] B. Chamberlain, J. Rowbottom, M. I. Gorinova, S. Webb, E. Rossi, M. M. Bronstein, Grand: Graph neural diffusion, *arXiv preprint arXiv:2106.10934* ArXiv (Cornell University) (2021). arXiv:2106.10934, doi:10.48550/arxiv.2106.10934.
- [9] E. Farhi, S. Gutmann, Quantum computation and decision trees, *Physical Review A* 58 (2) (1998) 915–928. doi:10.1103/PhysRevA.58.915.
- [10] S. Dernbach, A. Mohseni-Kabir, S. Pal, M. Gepner, D. Towsley, Quantum walk neural networks with feature dependent coins, *Applied Network Science* 4 (1) (2019). doi:10.1007/s41109-019-0188-2.

- [11] G. Verdon, T. McCourt, E. Luzhnica, V. Singh, S. Leichenauer, J. D. Hidary, Quantum graph neural networks, arXiv preprint arXiv:1909.12264ArXiv (Cornell University) (2019). **arXiv:** 1909.12264, doi:10.48550/arxiv.1909.12264.
- [12] Y. Zhan, Z. Gao, J. Li, Y. Liu, H. sun, Continuous-time quantum walks based graph neural network, arXiv preprint arXiv:2608.20738 (2026). **arXiv:** 2608.20738.
- [13] Z.-L. Li, W. Yu, Y. Wu, C. Wang, Ctqwformer: A ctqw-based transformer for graph classification, arXiv preprint arXiv:2605.09486ArXiv.org (2026). **arXiv:** 2605.09486, doi: 10.48550/arXiv.2605.09486.
- [14] G. Liu, J. Zhao, H. Wei, Z. Zhou, Y. Song, X. Zhou, G. Qi, Graphqwalk: Learning structural node embeddings via continuous quantum walk, IEEE Transactions on Network Science and Engineering 13 (2026) 4779–4796. doi:10.1109/tNSE.2025.3644803.
- [15] M. Defferrard, X. Bresson, P. Vandergheynst, Convolutional neural networks on graphs with fast localized spectral filtering, arXiv preprint arXiv:1606.09375ArXiv (Cornell University) (2016). **arXiv:** 1606.09375, doi:10.48550/arxiv.1606.09375.
- [16] Q. Li, Z. Han, X.-M. Wu, Deeper insights into graph convolutional networks for semi-supervised learning, arXiv preprint arXiv:1801.07606ArXiv (Cornell University) (2018). **arXiv:** 1801.07606, doi:10.48550/arxiv.1801.07606.
- [17] K. Oono, T. Suzuki, Graph neural networks exponentially lose expressive power for node classification, arXiv preprint arXiv:1905.10947ArXiv (Cornell University) (2019). **arXiv:** 1905.10947, doi:10.48550/arxiv.1905.10947.
- [18] T. K. Rusch, M. M. Bronstein, S. Mishra, A survey on oversmoothing in graph neural networks, arXiv preprint arXiv:2303.10993ArXiv (Cornell University) (2023). **arXiv:** 2303.10993, doi: 10.48550/arxiv.2303.10993.
- [19] F. Wu, T. Zhang, A. H. Souza, C. Fifty, T. Yu, K. Q. Weinberger, Simplifying graph convolutional networks, arXiv preprint arXiv:1902.07153ArXiv (Cornell University) (2019). **arXiv:** 1902.07153, doi:10.48550/arxiv.1902.07153.
- [20] J. Gasteiger, A. Bojchevski, S. Günnemann, Predict then propagate: Graph neural networks meet personalized pagerank, arXiv preprint arXiv:1810.05997ArXiv (Cornell University) (2018). **arXiv:** 1810.05997, doi:10.48550/arxiv.1810.05997.
- [21] M. Chen, Z. Wei, Z. Huang, B. Ding, Y. Li, Simple and deep graph convolutional networks, arXiv preprint arXiv:2007.02133ArXiv (Cornell University) (2020). **arXiv:** 2007.02133, doi: 10.48550/arxiv.2007.02133.
- [22] U. Alon, E. Yahav, On the bottleneck of graph neural networks and its practical implications, arXiv preprint arXiv:2006.05205ArXiv (Cornell University) (2020). **arXiv:** 2006.05205, doi: 10.48550/arxiv.2006.05205.
- [23] J. Topping, F. D. Giovanni, B. P. Chamberlain, X. Dong, M. M. Bronstein, Understanding over-squashing and bottlenecks on graphs via curvature, arXiv preprint arXiv:2111.14522ArXiv (Cornell University) (2021). **arXiv:** 2111.14522, doi:10.48550/arxiv.2111.14522.

- [24] M. Eliasof, E. Haber, E. Treister, Pde-gcn: Novel architectures for graph neural networks motivated by partial differential equations, arXiv preprint arXiv:2108.01938 Neural Information Processing Systems (2021). [arXiv:2108.01938](#).
- [25] T. K. Rusch, B. P. Chamberlain, J. Rowbottom, S. Mishra, M. M. Bronstein, Graph-coupled oscillator networks, arXiv preprint arXiv:2202.02296 (2022). [arXiv:2202.02296](#).
- [26] J. Kempe, Quantum random walks: An introductory overview, Contemporary Physics 44 (4) (2003) 307–327. [doi:10.1080/00107151031000110776](#).
- [27] O. Mülken, A. Blumen, Continuous-time quantum walks: Models for coherent transport on complex networks, Physics Reports 502 (2-3) (2011) 37–87. [doi:10.1016/j.physrep.2011.01.002](#).
- [28] A. M. Childs, Universal computation by quantum walk, Physical Review Letters 102 (18) (2009). [doi:10.1103/PhysRevLett.102.180501](#).
- [29] W. Liang, F. Yan, A. M. Ilyasu, A. S. Salama, K. Hirota, A hadamard walk model and its application in identification of important edges in complex networks, Computer Communications 193 (2022) 378–387. [doi:10.1016/j.comcom.2022.07.045](#).
- [30] M. Handley, Delay is not an option, in: Proceedings of the 17th ACM Workshop on Hot Topics in Networks, 2018, pp. 85–91. [doi:10.1145/3286062.3286075](#).
- [31] S. Kassing, D. Bhattacharjee, A. B. Águas, J. E. Saethre, A. Singla, Exploring the "internet from space" with hypatia, in: Proceedings of the ACM Internet Measurement Conference, 2020, pp. 214–229. [doi:10.1145/3419394.3423635](#).
- [32] O. Shchur, M. Mumme, A. Bojchevski, S. Günnemann, Pitfalls of graph neural network evaluation, arXiv preprint arXiv:1811.05868 (2018). [arXiv:1811.05868](#).
- [33] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, D. Meger, Deep reinforcement learning that matters, arXiv preprint arXiv:1709.06560 AAAI Conference on Artificial Intelligence (2017). [arXiv:1709.06560](#), [doi:10.1609/aaai.v32i1.11694](#).
- [34] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Sepah, E. Raff, K. Madan, V. Voleti, S. E. Kahou, V. Michalski, D. Serdyuk, T. Arbel, C. Pal, G. Varoquaux, P. Vincent, Accounting for variance in machine learning benchmarks, arXiv preprint arXiv:2103.03098 ArXiv (Cornell University) (2021). [arXiv:2103.03098](#), [doi:10.48550/arxiv.2103.03098](#).
- [35] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d’Alché-Buc, E. B. Fox, H. Larochelle, Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program), arXiv preprint arXiv:2003.12206 ArXiv (Cornell University) (2020). [arXiv:2003.12206](#), [doi:10.48550/arxiv.2003.12206](#).
- [36] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, arXiv preprint arXiv:1412.6980 UvA-DARE (University of Amsterdam) (2014). [arXiv:1412.6980](#), [doi:10.48550/arxiv.1412.6980](#).